

RAVEL: Reversible SLO-Aware Virtual Placement for Geo-Distributed LLM Serving

Abstract

Regional capacity constraints and rising LLM inference demand are driving providers to pool GPU resources across geographically distributed clusters, where request routing must navigate heterogeneous WAN paths and serving conditions. Together with heterogeneous request-level SLOs, these differences make placement feasibility request-specific: a region that is optional for one request may be the only SLO-feasible choice for another. Because SLO-feasible regions differ across requests, an arrival-time placement does more than balance load: it may consume capacity that is substitutable for the current request but indispensable to a later one. Once finalized, such an allocation cannot be corrected when later arrivals reveal its opportunity cost, even if the earlier request has not yet entered the serving engine. A natural response is to delay commitment. We find that fixed-delay late binding initially improves SLO attainment but eventually reverses the gain as the delay grows. Our diagnosis shows that the dominant recoverable loss is not the direct cost of waiting, but withholding already-arrived demand from subsequent capacity planning. This leads to RAVEL’s key insight: each request should influence capacity planning from the moment it arrives, while its placement remains revisable until engine handoff. RAVEL realizes this principle through tentative placement, immediate prospective demand accounting, and bounded pre-handoff replanning. On a vLLM deployment with five serving replicas across three clusters connected by physical WAN paths, RAVEL improves request-level SLO attainment over the best baseline at $16\times$ load by 8.8, 11.2, and 13.4 percentage points on LMSYS, Burst, and DeepResearch, respectively, while reducing P95 TTFT by 49.1–80.5% and P95 end-to-end latency by 15.5–56.8%.

1 Introduction

Geo-distributed LLM serving expands available GPU capacity, but not every region is equally useful to every request. Large-scale private WANs expose heterogeneous inter-region

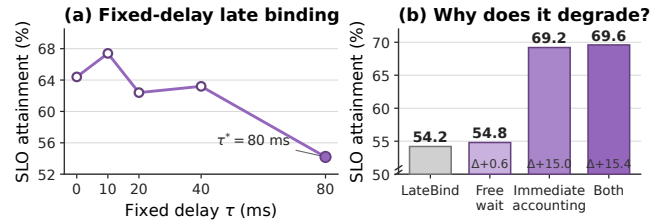


Figure 1: **Why fixed-delay late binding fails.** (a) Short deferral can improve SLO attainment, but longer waiting reverses the gain. (b) At $\tau = 80$ ms, eliminating the direct waiting penalty recovers only 0.6 pp, whereas immediate prospective accounting recovers 15.0 pp. The dominant recoverable loss therefore comes from delaying demand visibility together with commitment.

paths [11, 13], while distributed inference systems must additionally contend with heterogeneous accelerator, serving, and network conditions [5, 26, 34, 43, 48]. Together with heterogeneous request-level SLOs, these differences make placement feasibility request-specific: a region that is optional for one request may be the only SLO-feasible choice for another. Requests may impose different TTFT, TBT, or end-to-end completion objectives [22, 24, 44, 56, 58]. Regional capacity therefore has request-dependent opportunity cost: assigning a region to one request may consume capacity that another request cannot substitute away from.

This asymmetry makes an otherwise reasonable placement costly when later demand arrives. Existing distributed routers increasingly improve *where* an arriving request should execute using serving load, cache locality, WAN conditions, request-level SLOs, and adaptive feedback [19, 24, 39, 43, 48, 53]. However, they largely couple destination selection with commitment: once a request is placed using the currently visible state, later arrivals generally cannot change that placement before execution begins. Consider two regions, *A* and *B*. Request r_1 can satisfy its SLO in either region, whereas a later request r_2 can succeed only in *A*. Placing r_1 in the currently faster *A* is locally reasonable, but consumes r_2 ’s only feasible

region; redirecting r_1 to B allows both requests to succeed. This opportunity also appears in execution traces. In a trace-level decomposition of the Burst $12\times$ mechanism run, our conservative criterion confirms pre-handoff rescue events for at least 41.25% of misses under frozen placement: same-plan reassignment makes the affected request predicted feasible, and the request subsequently meets its SLO. The missing question is therefore not only *where should this request execute?*, but also *when should an earlier placement become irreversible?*

A natural response is to delay commitment so that later arrivals can influence placement. Figure 1(a) shows that fixed-delay late binding initially helps: short deferral exposes additional demand and improves SLO attainment. The benefit is not monotonic, however, and eventually reverses; at $\tau = 80$ ms, SLO attainment falls to 54.2%. Thus, preserving placement flexibility for longer is not by itself sufficient.

The dominant recoverable loss comes from delaying demand visibility rather than from the direct SLO-budget cost of waiting. Figure 1(b) isolates these effects at the same $\tau = 80$ ms operating point. An oracle that removes the artificial hold from SLO accounting while preserving the same late-binding behavior raises attainment only from 54.2% to 54.8%, a 0.6-percentage-point recovery. By contrast, preserving the same 80-ms final-binding delay while making held requests immediately visible to prospective capacity accounting raises attainment to 69.2%, recovering 15.0 percentage points. Fixed-delay late binding therefore preserves useful placement flexibility while withholding already-arrived demand from subsequent planning, leaving later requests with an incomplete view of prospective replica load. The dominant failure is thus not delay alone, but delaying demand visibility together with commitment.

RAVEL’s key insight is that deferred commitment does not require deferred accounting. A request can influence capacity planning from the moment it arrives while its tentative placement remains revisable until engine handoff. On arrival, RAVEL obtains replica-specific SLO quotes, selects a tentative serving replica, and immediately attributes the request’s prospective demand to that replica. While the request remains Router-owned, newly revealed demand or serving-state changes may trigger bounded replanning; reassignment transfers both tentative ownership and its prospective charge. Only successful engine handoff makes placement final. This lifecycle keeps tentative demand visible throughout while preserving the ability to correct stale pre-handoff allocations.

Turning this separation into a practical serving system requires four capabilities. First, the Router must determine which replicas remain SLO-feasible under heterogeneous WAN delay, engine backlog, Prefill–Decode interference, and request-specific success criteria. Second, tentative work must remain visible exactly once as placement changes, requiring prospective accounting that moves atomically with ownership. Third, newly observed information must be exploited without

unbounded waiting or repeatedly reconsidering all outstanding work, requiring bounded replanning. Finally, commitment and local execution must preserve the feasibility assumptions used by geographic placement while handling handoff failures safely. RAVEL realizes these requirements with calibrated replica-specific SLO quotes, transferable prospective accounting, a bounded Router-owned replanning cohort, an explicit engine-handoff commitment boundary, and engine-local TBT protection.

We implement RAVEL as an asynchronous routing layer for vLLM [14] and evaluate it on five serving replicas across three clusters connected by physical WAN paths. At $16\times$ load, RAVEL improves request-level SLO attainment over DualMap by **8.8, 11.2, and 13.4 percentage points** on LM-SYS, Burst, and DeepResearch, respectively, while reducing P95 TTFT by **49.1–80.5%** and P95 end-to-end latency by **15.5–56.8%**. Mechanism ablations reinforce the same design logic: removing prospective accounting lowers SLO attainment by **21.4 percentage points**, disabling pre-handoff reassignment lowers it by **13.2 points**, and removing engine-local TBT protection lowers it by **9.8 points**. Online replanning remains millisecond-scale, with a measured P95 scheduling-pass latency of 2.10 ms on Burst $12\times$.

In summary, this paper makes three contributions:

- We identify the request-dependent opportunity cost of regional capacity and show that premature commitment creates avoidable allocation-induced SLO misses. We further diagnose fixed-delay late binding and find that its dominant recoverable loss comes from delaying already-arrived demand visibility rather than from the direct waiting penalty.
- We introduce an immediate-accounting, deferred-commitment abstraction: tentative demand influences capacity planning from arrival while placement remains revisable until engine handoff. RAVEL realizes this abstraction through replica-specific SLO quotes, transferable prospective accounting, bounded pre-handoff replanning, and a failure-safe commitment boundary.
- We implement RAVEL on vLLM and evaluate it on a physical-WAN deployment, demonstrating higher request-level SLO attainment across conversational, bursty, and completion-oriented workloads with millisecond-scale online planning overhead.

2 Problem and Design Opportunity

Geo-distributed request placement is an online allocation problem in which demand visibility and placement finality need not occur at the same time. For request i , let t_i^{vis} denote the time at which the request first contributes to the capacity state used for subsequent placement decisions, and let t_i^{final} denote the time after which the routing layer can no longer

change its destination. A placement associates the request with a candidate serving replica, whereas visibility determines when its prospective demand influences later decisions. These are logically distinct lifecycle events. Because future arrivals are unknown, the router must choose placements and their finality using only causally available demand and serving state, with the goal of maximizing the fraction of requests that satisfy their applicable request-level SLOs.

2.1 Request-Dependent Regional Feasibility

SLO feasibility is specific to a request, replica, and decision time. Prior SLO-aware serving systems show that whether a request can meet its objective depends on request-specific requirements and evolving serving conditions [22, 24, 56, 58]. At the geographic placement layer, these conditions additionally include the WAN path to each candidate region.

Let c denote a serving region, $\mathcal{R}_c$ the replicas in that region, and $r \in \mathcal{R}_c$ a candidate replica. For request i , let $\Phi_{i,r}(t)$ indicate whether replica r is predicted, using only information available at time t , to satisfy every SLO constraint applicable to i . The prediction depends on the request’s SLO semantics and remaining slack, estimated service demand, the WAN path to region c , and the observed serving state at replica r . Because these inputs evolve over time, $\Phi_{i,r}(t)$ is an online feasibility estimate rather than a static property of a replica or region. Section 3.2 describes the estimator used by RAVEL.

The feasible-region set captures the geographic alternatives available to a request at a given decision time:

$$\mathcal{F}_i(t) = \{c \mid \exists r \in \mathcal{R}_c : \Phi_{i,r}(t) = 1\}. \quad (1)$$

The time index captures two sources of change. First, new arrivals and accounting updates alter the serving state visible to the router, so contention may expand or shrink $\mathcal{F}_i(t)$. Second, elapsed time consumes TTFT or completion slack, so $\mathcal{F}_i(t)$ may shrink even if serving load and network conditions remain unchanged.

The size and composition of $\mathcal{F}_i(t)$ determine how substitutable regional capacity is for request i . A request with several feasible regions has alternatives, whereas one with a small feasible set depends on comparatively scarce regional capacity. For example, if $\mathcal{F}_1 = \{A, B\}$ and $\mathcal{F}_2 = \{A\}$, assigning request 1 to A consumes request 2’s only feasible option even though request 1 could execute in B . The opportunity cost of a region therefore depends on which requests can substitute away from it. An SLO failure can consequently be *allocation-induced* rather than necessarily *capacity-induced*: an earlier request may occupy capacity that it can replace elsewhere but a later request cannot.

Table 1: Separating demand visibility from placement finality.

Policy	Demand visible	Placement final
One-shot	At arrival	At arrival
LateBind- τ	After τ	After τ
Desired operating point	At arrival	Later

2.2 Coupling Demand Visibility with Commitment

Later arrivals can change the opportunity cost of a placement that was reasonable when it was made. The router cannot know such arrivals in advance, and we do not assume an arrival oracle or an offline-optimal allocation. This setting shares concerns with online and shared-state cluster scheduling, where decisions are made from incomplete and evolving information [10, 16, 30, 36]. Any practical policy must therefore exploit only requests and serving state that have become causally observable.

Demand visibility and placement finality are distinct lifecycle decisions. Table 1 illustrates three operating points. A one-shot policy makes an arriving request visible immediately but also finalizes its placement immediately, leaving no opportunity for later arrivals to revise it. Our diagnostic LateBind- τ policy instead holds the request centrally unassigned and unaccounted for τ , and then makes one final placement. Thus,

$$t_i^{\text{vis}} = t_i^{\text{final}} \quad (2)$$

under both policies; they differ mainly in when this coupled event occurs. The remaining operating point is to expose demand before placement becomes irreversible.

Fixed-delay late binding shows that preserving optionality is useful, but delaying commitment also introduces costs. Figure 1(a) shows that short deferral improves SLO attainment by exposing additional arrivals, while the gain eventually reverses as τ grows. One intuitive cost is direct slack loss: holding request i for τ consumes τ of its remaining TTFT or completion budget and can shrink $\mathcal{F}_i(t)$ even when serving load is unchanged. Figure 1(b), however, shows that this direct waiting cost is not the dominant recoverable penalty in our diagnostic.

The dominant recoverable loss comes from withholding already-arrived demand from subsequent capacity planning. At $\tau = 80$ ms, eliminating the direct waiting penalty recovers only 0.6 percentage points of SLO attainment. Keeping the same final-binding delay but immediately exposing held requests to prospective accounting instead recovers 15.0 percentage points. LateBind- τ therefore changes two lifecycle events simultaneously: postponing finality preserves the opportunity to revise placement, but postponing visibility causes later requests to plan against incomplete prospective demand.

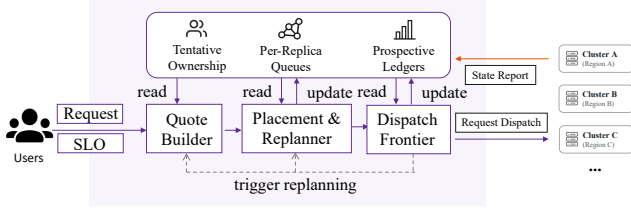


Figure 2: **RAVEL request lifecycle and ownership boundary.** On arrival, RAVEL constructs replica-specific SLO quotes, selects a tentative placement, and immediately accounts the request’s prospective demand. While Router-owned, the request remains eligible for bounded reassignment, with tentative ownership and prospective accounting transferred together. Capacity-gated dispatch initiates engine handoff; only successful handoff makes placement final. Live engine state feeds back into subsequent quotes and accounting.

The desired operating point therefore satisfies

$$t_i^{\text{vis}} < t_i^{\text{final}}, \quad (3)$$

so that arrived work affects capacity planning before its placement becomes irreversible.

Realizing this operating point safely requires more than simply delaying commitment. A practical design must (i) estimate request-specific SLO feasibility so that reassignment considers only viable alternatives; (ii) make tentative demand visible exactly once and transfer its prospective charge when placement changes; (iii) exploit newly observed demand without unbounded waiting or global replanning; and (iv) define a failure-safe commitment boundary after which placement no longer moves. Section 3 describes how RAVEL realizes these requirements and chooses a concrete commitment boundary.

3 RAVEL Design

3.1 Overview

RAVEL realizes the separation exposed by Figure 1: arrival establishes demand visibility, Router ownership preserves placement revisability, and successful engine handoff establishes finality. While Router-owned, each request has exactly one tentative replica owner and one prospective demand representation. Its destination may change as new demand or serving-state feedback arrives, but its demand remains visible throughout. After successful handoff, the serving engine owns the request and its geographic placement is no longer revised.

Figure 2 summarizes this lifecycle. On arrival, RAVEL constructs request-specific SLO quotes for candidate replicas, chooses an initial virtual placement, and immediately accounts the request’s prospective demand at that replica. Subsequent arrivals and engine feedback may trigger bounded replanning over a cohort of still-Router-owned requests. A

reassignment moves both tentative ownership and replica-dependent prospective demand to the new replica. Requests approach execution through a shallow capacity-gated dispatch frontier; a rejected handoff restores Router ownership, whereas a successful handoff transfers ownership to the serving engine and makes placement final.

The design maintains three classes of invariants. *Single representation*: each Router-owned request has exactly one tentative owner and one prospective demand representation. *Bounded mobility*: only Router-owned requests may move, and replanning is causal and bounded in both time and scope. *Explicit finality*: tentative placement, hold expiration, and dispatch eligibility do not commit a request; only successful engine handoff does. The following sections describe how RAVEL enforces these properties.

3.2 Replica SLO Quotes

Revisability is useful only if the Router can determine which replicas remain SLO-feasible for each request. Because regional capacity is request-dependent (Section 2.1), RAVEL represents each candidate using a request- and replica-specific SLO quote rather than a request-independent load score. A quote summarizes whether the request is predicted to satisfy its applicable SLOs, the request-specific objective used to rank viable targets, and secondary capacity-pressure information.

Let $c \in \mathcal{C}$ denote a serving cluster, $\mathcal{R}_c$ its replicas, $c(r)$ the cluster containing replica r , and $\mathcal{R} = \bigcup_c \mathcal{R}_c$. For request i and candidate replica r , RAVEL constructs

$$Q_{i,r}(t) = \left(\Phi_{i,r}(t), \hat{T}_{i,r}^{\text{obj}}(t), \mathbf{z}_{i,r}(t) \right), \quad (4)$$

where $\Phi_{i,r}(t)$ indicates whether all applicable SLO constraints are predicted to hold, $\hat{T}_{i,r}^{\text{obj}}(t)$ is the request-class-specific objective used for ordering and slack calculations, and $\mathbf{z}_{i,r}(t)$ contains secondary capacity-pressure signals. Quotes use only information causally available at time t and are online predictions rather than formal SLO guarantees.

RAVEL combines calibrated cluster-level service profiles with live and prospective replica state. At active Decode concurrency n , it models Prefill service for prompt size p as $P_{c,n}(p) = b_{c,n} + a_{c,n}p$ and uses $d_c(n)$ for calibrated post-first-token Decode time per token. For request i and replica r , let $\hat{S}_{i,r}^{\text{pre}}$ denote predicted Prefill-side service delay, including live engine Prefill debt, Router-owned prompt work ahead of i , the request’s own prompt demand, and tentative assignments made earlier in the same planning pass. RAVEL predicts

$$\widehat{\text{TTFT}}_{i,r}(t) = (t - a_i) + \rho_{o_i, c(r)} + \hat{S}_{i,r}^{\text{pre}}, \quad (5)$$

where a_i is request arrival time and $\rho_{o_i, c(r)}$ is directed WAN delay from origin o_i to cluster $c(r)$. The elapsed term directly

charges time already spent under Router ownership, so a previously feasible replica may become infeasible as request slack shrinks.

RAVEL evaluates each SLO according to its semantics rather than collapsing them into one latency score. For LATENCY requests, TTFT and TBT feasibility are checked independently: a lower predicted TTFT cannot compensate for a predicted TBT violation. For completion-oriented requests, RAVEL separates a conservative output-demand estimate used for feasibility from an expected estimate used for shared service-pressure accounting, avoiding both optimistic admission and systematic over-accounting. Appendix A.2 gives the complete TTFT, TBT, and completion equations, calibration procedure, runtime sources, and fallback rules.

3.3 Immediate and Transferable Demand Accounting

Figure 1 shows that placement revisability loses much of its value when already-arrived work is withheld from capacity accounting. RAVEL therefore makes tentative demand visible immediately and exactly once, without making its placement final. Every Router-owned request is associated with one tentative replica queue and one set of replica-dependent prospective charges.

When request i receives a virtual placement at replica r , it immediately enters r 's Router-owned queue. Its prompt contributes to the queue-ahead Prefill work observed by later quotes, including quotes constructed later in the same planning pass. The Router also maintains lightweight prospective completion-service and LATENCY/TBT pressure signals. These signals complement, rather than replace, the replica's live pending and running engine state.

Replanning must preserve this visibility as placement changes. If i moves from replica r to r' , RAVEL atomically transfers its tentative queue ownership and recomputes the replica-dependent prospective charges for r' . The serialized control path ensures that a planning snapshot observes request i at exactly one tentative owner—never at both replicas and never at neither. The destination then exposes the transferred demand to all subsequent quote and placement decisions.

Visibility also remains continuous across the ownership boundary. Successful handoff ends Router queue ownership, after which committed work is represented by live engine state and an active-service charge until terminal completion. A rejected handoff restores the Router-owned prospective state. These transitions establish RAVEL's central accounting invariant: *tentative work is immediately visible, but visibility does not make its placement final*. Appendix A.3 gives exact charge definitions, transfer operations, deduplication rules, lifecycle transitions, and stale-callback protection.

3.4 Bounded Pre-Handoff Replanning

Later arrivals can expose stale allocations, but exploiting them requires revisability to remain bounded in both time and scope. RAVEL therefore reconsiders only Router-owned requests whose placements can still change. Committed work remains fixed but continues to affect later quotes through live engine state. Planning is event-driven, serialized, and coalesced so that each pass operates on a consistent prospective view.

Bounding the revisable interval. For selected non-LATENCY requests, RAVEL preserves a short observation window before dispatch. Importantly, the request is already tentatively placed and prospectively accounted during this interval; the hold changes only how long the placement remains revisable.

Let

$$\sigma_i^{\text{arr}} = \max\left\{0, \Delta_i - \min_r \widehat{T}_{i,r}^{\text{obj}}(a_i)\right\}$$

denote the request's arrival-time planning slack, where Δ_i is its applicable TTFT or completion planning budget. RAVEL sets

$$h_i = \begin{cases} \min\{\rho_i^{\min}, \sigma_i^{\text{arr}}\}, & \text{if } i \text{ is hold-eligible,} \\ 0, & \text{otherwise,} \end{cases} \quad (6)$$

where $\rho_i^{\min}$ is the minimum positive RTT to a remote serving cluster. Thus, deliberate observation is bounded by both a network timescale and the request's available planning slack. LATENCY requests use $h_i = 0$. Expiration of h_i makes the request dispatch-eligible; it neither removes its prospective accounting nor commits its placement.

Bounding replanning scope. Each planning pass considers a deterministic deadline-first prefix of at most K movable requests. Requests outside the cohort retain their tentative owners and enter the planning snapshot as fixed context. Within the cohort, each selected placement immediately updates pass-local prospective state before the next request is evaluated. With $R = |\mathcal{R}|$ candidate replicas and effective cohort bound K_{eff} , one greedy pass performs $O(K_{\text{eff}}R)$ request-replica quote comparisons. Under sustained completion pressure, RAVEL may expand the effective cohort within the deployment's finite sequence capacity; this changes how much Router-owned work is reconsidered, not the placement or commitment semantics.

Choosing initial and revised placements. RAVEL uses the same SLO-aware placement rule for an arrival and for requests reconsidered during replanning. If at least one replica is predicted feasible, the Router selects among feasible targets. If none is feasible, it first minimizes normalized predicted SLO violation. Remaining choices are ordered lexicographically by *SLO safety*, *capacity pressure*, and finally *efficiency*.

Algorithm 1: RAVEL Router Control Loop

```
1 Function ONARRIVAL( $i$ ):
2    $Q_i \leftarrow \text{QUOTEREPLICAS}(i)$ 
3    $r_i \leftarrow \text{SLOAWAREPLACE}(i, Q_i)$ 
4    $\text{ASSIGNANDACCOUNT}(i, r_i)$ 
   // Visible immediately; placement remains tentative
5    $h_i \leftarrow \text{MOBILITYHOLD}(i, Q_i)$ 
6   if  $h_i = 0$  then
7      $\text{REQUESTREPLAN}()$ 
8   else
9      $\text{ARMREPLAN}(h_i)$ 

10 Function REPLAN():
11    $C \leftarrow \text{BOUNDEDMOVABLECOHORT}()$ 
   // Only Router-owned requests are movable
12   for  $i \in C$  do
13      $Q_i \leftarrow \text{QUOTEREPLICAS}(i)$ 
14      $r'_i \leftarrow \text{SLOAWAREPLACE}(i, Q_i)$ 
15      $\text{TRANSFERORREFRESH}(i, r'_i)$ 
   // Update prospective state before the next request

16 Function DISPATCH( $i$ ):
17   if  $\text{HELD}(i)$  or  $\text{FRONTIERBLOCKS}(i)$  then
18     return
19    $r_i \leftarrow \text{TENTATIVEOWNER}(i)$ 
20    $s \leftarrow \text{ATTEMPTHANDOFF}(i, r_i)$ 
21   if  $s = \text{REJECTED}$  then
22      $\text{RESTOREROUTEROWNERSHIP}(i, r_i)$ 
23   else if  $s = \text{COMMITTED}$  then
24      $\text{FINALIZEPLACEMENT}(i, r_i)$ 
   // Successful handoff is the commitment boundary
```

and stability. The capacity-pressure tier discourages broadly substitutable requests from occupying scarce SLO-feasible capacity, while lower-priority signals include predicted service cost, movement, prefix locality, and deterministic tie-breaking. Consequently, locality or stability cannot override an avoidable predicted SLO miss.

Replanning applies this same rule to the bounded movable cohort. After each target is selected, RAVEL immediately updates pass-local prospective state so that later requests in the same pass observe the demand created by earlier tentative choices. Algorithm 1 summarizes the resulting Router control loop. Appendix A.4 and Appendix A.5 provide the exact trigger predicates, eligibility rules, cohort ordering, reassignment limits, and placement key.

3.5 Commitment and Execution Protection

Revisability must end at a precise boundary, and execution after that boundary must preserve the assumptions under which the placement was judged feasible. RAVEL therefore separates approaching commitment, committing ownership, and protecting execution after handoff.

Capacity-gated dispatch. A shallow dispatch frontier limits how much Router-owned work approaches commitment at once. The frontier determines *when* a tentatively placed request may be handed to its engine; it does not choose *where* the request executes. Work behind the frontier remains tentatively placed, prospectively accounted, and eligible for replanning. A request becomes dispatch-eligible when its tentative replica has sufficient engine headroom, bounding proximity to commitment without introducing a separate placement or admission policy. Appendix A.7 gives the frontier construction and headroom test.

Engine handoff as the commitment boundary. Commitment is an ownership transition rather than a timer or queue event. For a dispatch-eligible request, RAVEL attempts replica registration followed by creation of the engine-facing posting task. Only successful completion of the handoff transfers the request from Router-owned, revisable state to engine-owned, final state. Failures before this boundary restore or clean the Router-owned state; failures after handoff terminate the committed attempt without reopening geographic placement. Attempt identifiers prevent stale callbacks from modifying newer ownership or accounting state. Thus, tentative placement, hold expiration, dispatch eligibility, and temporary removal from a Router queue are all non-commitment events.

Preserving quote assumptions at the engine. Final placement remains useful only if local execution preserves the service conditions under which that placement was considered feasible. In particular, Prefill–Decode interference can invalidate a TBT-feasible quote after handoff. When Chunked Prefill is enabled and LATENCY Decode is active, RAVEL exports the tightest applicable TBT requirement and deadline metadata to a local serving adapter. The adapter combines this requirement with a calibrated conservative Prefill envelope to bound the next Prefill quantum and gives LATENCY requests deadline/TBT-aware local priority, while non-LATENCY work retains stable FCFS behavior. This mechanism protects the execution assumptions underlying replica quotes; it does not introduce another geographic placement decision.

Together, these mechanisms complete the lifecycle derived in Section 2: *arrival makes demand visible, Router ownership keeps placement revisable, and successful engine handoff makes placement final*. Appendix A.9 and Appendix A.8 give the complete ownership state machine, failure transitions, activation rules, and local scheduling details.

4 Implementation

Our implementation preserves the ownership, accounting, and commitment semantics defined in Section 3 while integrating RAVEL with an unmodified vLLM serving stack. We

implement the Router in Python as an asynchronous control layer in front of geo-distributed vLLM replicas, targeting vLLM 0.8.5.post1 and its frozen V0 scheduler contract.

Serialized Router state. Exactly-once ownership and accounting transfer require serialized Router state updates. All control-plane operations that modify Router-owned queues, perform replanning, or initiate dispatch execute under a single `asyncio` scheduler lock. New arrivals, mobility-hold expirations, engine feedback, and explicit replanning requests therefore enter the same serialized scheduling path, preventing concurrent passes from racing on tentative ownership or prospective accounting. Terminal callbacks may asynchronously release tracked service charges, but any resulting replanning re-enters the same serialized path. This serialization ensures that ownership transfer and prospective accounting updates are observed atomically by every planning pass.

Calibrated service profiles. RAVEL derives replica-specific SLO quotes from deployment-calibrated service models rather than hard-coded token rates. RAVEL derives serving estimates from externally calibrated schema-v2 profiles, each bound to a serving cluster and carrying an engine fingerprint; formal runs require a valid profile for every cluster. At runtime, RAVEL linearly interpolates Prefill and Decode measurements between calibrated concurrency points and clamps queries outside the measured range to the nearest endpoint. The engine adapter also constructs a conservative Prefill envelope from the maximum calibrated slope and intercept across concurrency levels. For multi-stage completion requests, the Router may additionally use an offline semantic output profile indexed only by request-visible workflow metadata, such as (*stage_id*, *stage_num*). These profiles expose neither future arrivals nor realized output lengths, and therefore provide deployment-specific inputs to Section 3.2 without violating RAVEL’s causal information model.

vLLM integration and local TBT protection. Quote validity requires local execution to preserve the service assumptions used by the Router. RAVEL integrates with vLLM without modifying its source tree. At process startup, a lightweight adapter feature-detects and wraps `Scheduler._get_priority` and `Scheduler._schedule_chunked_prefill`; if either required hook is unavailable, startup fails rather than silently disabling the corresponding local controls. For LATENCY requests, the adapter encodes a Router-derived deadline-order key and the applicable TBT requirement in a reserved priority range, while non-LATENCY requests retain ordinary FCFS priority in the mixed path. When LATENCY Decode is active, the adapter combines the tightest active TBT requirement with the conservative Prefill envelope to derive a block-aligned bound on the next Prefill

quantum. It temporarily reduces `max_num_batched_tokens` for that Chunked-Prefill scheduling iteration and restores the configured value in a `finally` block. The mechanism therefore protects the execution assumptions underlying replica-level quotes without introducing another Router-level geographic placement policy.

Failure-safe asynchronous handoff. RAVEL implements deferred commitment as a failure-safe asynchronous ownership boundary. After Router-side admission and budget checks succeed, the target replica first registers the request, after which RAVEL creates and tracks the corresponding engine-facing posting task. Successful replica registration together with successful posting-task creation constitutes the handoff defined in Section 3.5; only then does geographic placement become final.

If registration is rejected, RAVEL restores the request to its Router-owned queue together with its prospective state, leaving it eligible for ordinary mobility and replanning. Failures while creating or executing an engine-facing posting task are surfaced to the control plane rather than silently converted into a new geographic placement. Posting attempts carry attempt identifiers so that delayed callbacks from an older attempt cannot modify ownership or accounting associated with a newer one.

After successful handoff, replica-reported pending and running state continues to inform subsequent Router quotes, while the corresponding active-service charge remains visible until terminal completion. At termination, RAVEL releases that charge and requests another serialized scheduling pass so that newly available capacity can affect requests that remain Router-owned. Thus, the implementation preserves the lifecycle established in Section 3: Router ownership preserves revisability, successful handoff establishes finality, and committed work remains visible through live engine state.

5 Evaluation

We evaluate RAVEL along five dimensions. **RQ1:** Does RAVEL improve request-level SLO attainment as offered load increases? **RQ2:** Do these gains persist across repeated runs? **RQ3:** How much do immediate prospective accounting, pre-handoff revisability, and engine-local TBT protection each contribute to the end-to-end gains? **RQ4:** Are replica-specific quotes accurate enough to support request-level feasibility decisions? **RQ5:** Is bounded replanning fast enough for the online serving path?

The main result matrix reports one 500-request replay per policy–workload–load cell. A separate five-run campaign evaluates replay-to-replay variation at 16× load. We report these two experimental contexts separately and do not average their results together.

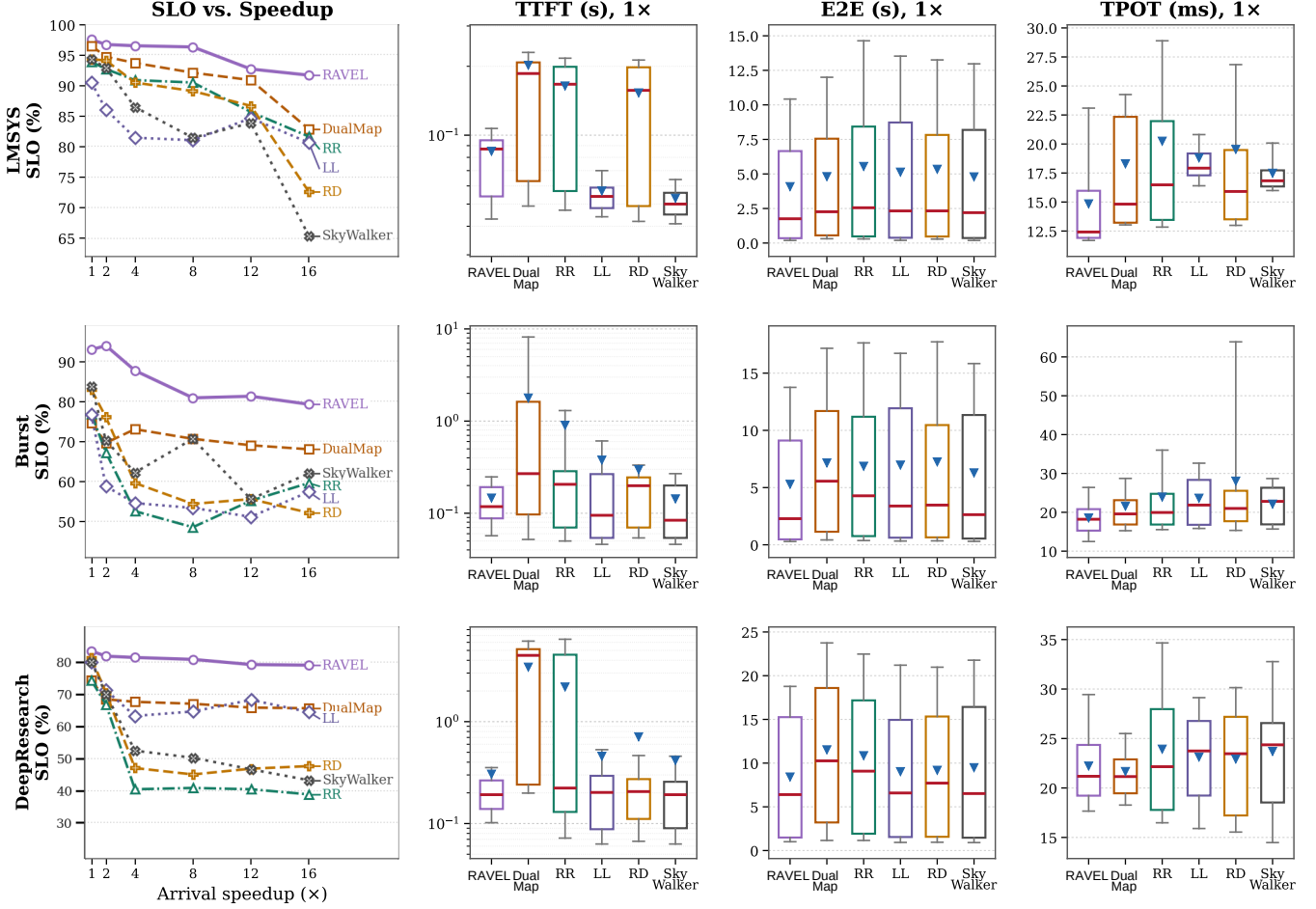


Figure 3: End-to-end results for LMSYS (top), Burst (middle), and DeepResearch (bottom), using one 500-request replay per policy–workload–load cell. The first column reports request-level SLO attainment from $1\times$ to $16\times$ load. The remaining columns report per-request TTFT, E2E, and TPOT distributions at $1\times$, isolating low-load latency behavior when capacity is comparatively plentiful. In each box plot, the red line marks the median, the box spans the 25th–75th percentiles, the whiskers show the 10th–90th percentiles, and the inverted triangle marks the mean.

5.1 Experimental Setup

Testbed. We evaluate all policies on the same five-replica, three-cluster physical-WAN deployment. The replicas are distributed across the clusters in a 2/2/1 configuration. Each replica serves Qwen3-1.7B on one NVIDIA RTX 4090 using vLLM 0.8.5.post1’s V0 scheduler with bfloat16 weights. The directed physical WAN paths have measured RTTs ranging from 10.1 to 37.9 ms. Table 2 summarizes the common engine and replay settings.

Workloads and SLOs. The three workloads stress different sources of online placement uncertainty. *LMSYS* represents heterogeneous conversational traffic. *Burst* preserves the request contents of its source trace but replaces arrival timestamps with a bursty schedule, stressing decisions made from rapidly changing queue state. *DeepResearch* represents multi-

stage completion workflows whose output demand depends on request-visible stage metadata.

Each request carries an SLO class and a request-specific multiplier. The base budgets are 0.8 s for TTFT, 80 ms for TBT, and 8 s for completion latency, after which the configured multiplier is applied. A LATENCY request succeeds only if it satisfies both its TTFT bound and every TBT bound. THROUGHPUT and COLLECTIVE requests succeed if their end-to-end completion latency remains within the corresponding completion budget.

Replay protocol. We vary contention by compressing the original inter-arrival timeline while preserving request order, contents, output metadata, and SLOs. At load factor x , every original inter-arrival interval is divided by x . Before each cell, we reset replica prefix caches and start a fresh policy run.

Table 2: Common experimental configuration.

Component	Configuration
Serving topology	5 replicas in 3 clusters (2/2/1)
Model and hardware	Qwen3-1.7B; one RTX 4090 per replica
Serving engine	vLLM 0.8.5.post1 V0; bfloat16
Maximum model length	16,384 tokens
Engine capacity	64 sequences; 16,384 batched tokens
KV-cache configuration	Block size 16; automatic prefix caching
Directed WAN RTT	10.1–37.9 ms
Requests per main cell	500; seed 42
Offered-load factors	1, 2, 4, 8, 12, 16×

All policies receive the same request-visible information and run under the same model, topology, WAN paths, and replay schedule. The main matrix contains one 500-request replay per cell. The repeated-run campaign is conducted separately using five paired replays at 16×

Policies and metrics. We compare RAVEL with RoundRobin (RR), LeastLoad (LL), RD, DualMap, and SkyWalker. For the single-run headline results, numerical gains are reported relative to DualMap. The repeated-run campaign follows its designated best-baseline protocol.

Our primary metric is request-level SLO attainment. Secondary metrics are time to first token (TTFT), end-to-end latency (E2E), and time per output token (TPOT). TTFT and E2E are measured from request arrival at the Router and therefore include Router-owned time, WAN delay, queueing, and engine execution.

5.2 End-to-End Performance

RAVEL’s advantage grows with contention, when decisions based only on arrival-time state are more likely to become stale before execution. Figure 3 shows that RAVEL maintains the highest request-level SLO attainment across the evaluated load sweep. The separation becomes most visible at high load, where rapidly changing prospective queue state increases the cost of committing a geographic placement too early.

At 16×

The 1×

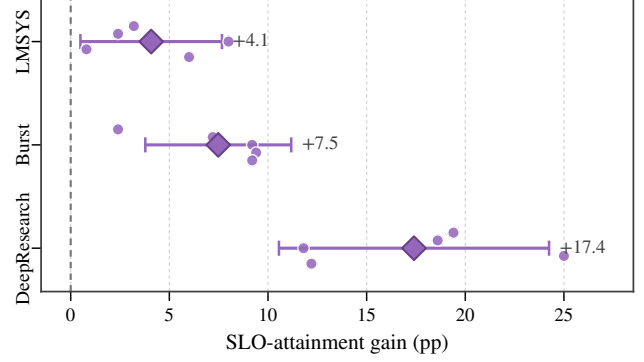


Figure 4: **Repeated-run robustness at 16×** load. Small circles show paired RAVEL–best-baseline SLO-attainment differences for five 500-request replays per workload; diamonds and whiskers show the mean and 95% confidence interval. Mean gains are 4.1, 7.5, and 17.4 percentage points for LMSYS, Burst, and DeepResearch, respectively.

overlapping latency distributions also illustrate why raw latency alone is not the serving objective. A request with sufficient slack may tolerate a different placement if doing so preserves scarce SLO-feasible capacity for a more constrained request. RAVEL therefore optimizes the fraction of requests satisfying their own SLOs, rather than minimizing every latency sample independently.

5.3 Repeated-Run Robustness

The load sweep establishes behavior across workloads and contention levels, while a separate paired campaign tests replay-to-replay robustness at 16×

The repeated runs preserve RAVEL’s advantage on all three workloads. Mean gains are 4.1 percentage points on LMSYS, 7.5 points on Burst, and 17.4 points on DeepResearch. For each workload, the 95% confidence interval over the five paired run-level differences lies above zero. This supports the conclusion that the average improvement persists under replay variation, without implying that every request or every possible replay must improve.

These five-run values are separate from the single-run matrix in Figure 3. They come from a different experimental batch and use a different statistical unit. For example, the 17.4-point DeepResearch repeated-run mean should not be substituted for the 13.4-point single-run headline result. Appendix C reports the individual paired values and statistical construction.

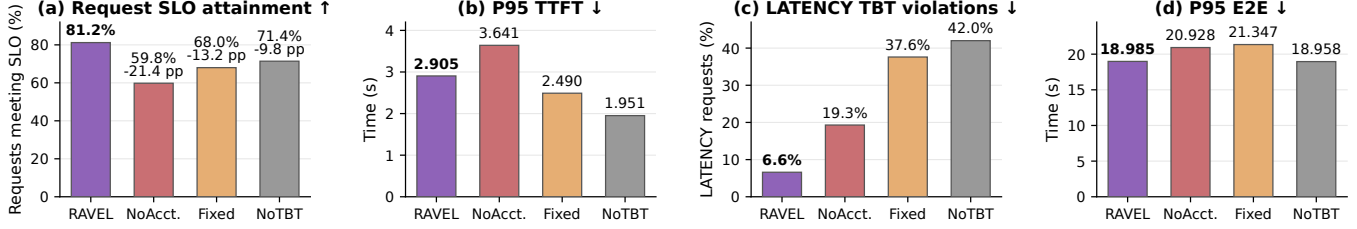


Figure 5: **Mechanism ablation on the 500-request Burst workload at 12 $\times$ load.** NoAccounting removes prospective accounting of Router-owned tentative demand while retaining quotes, replanning, and the TBT guard. FixedPlacement retains prospective accounting and the TBT guard but disables pre-handoff reassignment. NoTBTGuard retains global routing, accounting, and replanning but disables engine-local TBT protection. The panels report (a) request-level SLO attainment, (b) P95 TTFT, (c) the TBT-violation rate among LATENCY requests, and (d) P95 E2E. Higher is better only for SLO attainment.

5.4 Mechanism Contributions

We isolate RAVEL’s three main mechanisms on Burst at 12 $\times$, where burst-driven state changes are pronounced without the system collapsing into universal overload. *NoAccounting* retains replica-specific SLO quotes, pre-handoff replanning, and engine-local TBT protection, but excludes Router-owned tentative demand from the prospective state used by subsequent placement decisions. *FixedPlacement* retains RAVEL’s initial SLO-aware placement, prospective accounting, and TBT guard, but disables pre-handoff reassignment. *NoTBTGuard* retains global routing, prospective accounting, and replanning, but disables the engine-local Prefill control used to protect active LATENCY Decode requests.

Removing prospective accounting causes the largest attainment drop. Full RAVEL attains 81.2% request-level SLO attainment, compared with 59.8% for NoAccounting (–21.4 **percentage points**), 68.0% for FixedPlacement (–13.2 **points**), and 71.4% for NoTBTGuard (–9.8 **points**). The NoAccounting result closes the loop with Figure 1: allowing placement to remain revisable is not sufficient if tentative work disappears from the state used to make subsequent capacity decisions. Likewise, the FixedPlacement gap shows that immediate accounting alone cannot repair a stale geographic assignment once that assignment has been made irreversible.

The component-level metrics show that the three mechanisms protect different failure surfaces. NoAccounting increases P95 TTFT from 2.905 s to 3.641 s, raises the LATENCY-request TBT-violation rate from 6.6% to 19.3%, and increases P95 E2E from 18.985 s to 20.928 s. FixedPlacement obtains a lower P95 TTFT than RAVEL (2.490 versus 2.905 s), yet its TBT-violation rate rises to 37.6% and its P95 E2E increases to 21.347 s. NoTBTGuard reduces P95 TTFT further to 1.951 s and has nearly the same P95 E2E as RAVEL (18.958 versus 18.985 s), but its TBT-violation rate reaches 42.0%. Thus, low TTFT or E2E alone does not imply high request-level SLO attainment: prospective accounting, geographic revisability, and local TBT protection address complementary sources of failure.

A trace-level decomposition of the same Burst 12 $\times$ mech-

anism run explains why pre-handoff reassignment is useful. We conservatively classify a miss under frozen pre-handoff placement as a confirmed rescue only when the same planning episode makes the affected request change from predicted infeasible to predicted feasible through reassignment, and that request subsequently satisfies its realized SLO. Under this criterion, at least **41.25%** of frozen-placement misses are confirmed rescue events without adding serving capacity. The remaining misses are unresolved by this criterion rather than proven irrecoverable. This statistic is a trace-level decomposition of the same mechanism run, not a separate experimental campaign.

5.5 Quote Accuracy

Replica-specific quotes are useful only if they preserve the feasibility decision that drives placement and replanning. Across the selected final-handoff records in our 16 $\times$ quote analysis, the empirical false-feasible and false-infeasible rates are **6.2%** and **1.0%**, respectively.

For final-handoff record i , we define normalized quote error as

$$e_i = \frac{\hat{L}_i - L_i}{B_i},$$

where $\hat{L}_i$ is predicted latency, L_i is observed latency, and B_i is the corresponding request-specific SLO budget. For LATENCY requests, L_i denotes TTFT; for completion-oriented requests, L_i denotes E2E. Negative error is optimistic and positive error is conservative.

Figure 6 shows that most normalized errors remain near zero, although completion quotes exhibit a longer optimistic tail. More importantly, the quotes usually preserve the decision-relevant feasibility classification. Of the 1,672 records quoted as feasible, 104 later violate the corresponding SLO, yielding a 6.2% empirical false-feasible rate. Of the 828 records quoted as infeasible, only 8 complete feasibly, yielding a 1.0% empirical false-infeasible rate.

These measurements cover placements that RAVEL actually selected and handed off; they do not observe counter-

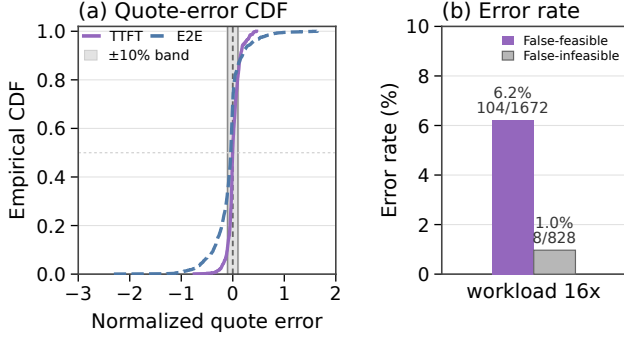


Figure 6: **Quote accuracy in the 16 $\times$ quote analysis.** (a) Empirical CDF of normalized quote error $(\hat{L}_i - L_i)/B_i$ for LATENCY (TTFT) and completion (E2E) requests; shading marks errors within $\pm 10\%$ of the applicable SLO budget. (b) Feasibility-classification error rates over the same $N = 2,500$ selected final-handoff records. False-feasible and false-infeasible rates are 6.2% and 1.0%, respectively.

factual execution on unselected replicas. The reported rates therefore characterize the empirical quality of the online decision signal rather than providing a formal guarantee that every feasible quote will satisfy its SLO.

5.6 Planner Overhead

Bounded replanning adds millisecond-scale latency on the actual online control path. Figure 7(b) reports scheduling-pass latency from the Burst 12 $\times$ mechanism run. Across 281 actual passes, median latency is 0.998 ms and P95 latency is 2.101 ms. The mean observed cohort contains 3.65 requests, its P95 is 7, and its maximum is 9, well below the configured online bound of $K = 64$.

A controlled benchmark measures the cost of approaching the configured cohort bound. At $K = 64$, median planner latency is 10.99 ms for mixed cohorts and 20.10 ms for completion-oriented cohorts. Increasing the cohort to the stress-only value $K = 128$ raises these values to 23.47 and 42.75 ms, respectively. The shaded ranges in Figure 7 show measurement dispersion across repetitions rather than confidence intervals.

The gap between the controlled bound and the much smaller cohorts observed at runtime explains why actual scheduling passes are substantially faster than the bound-oriented benchmark. These measurements characterize the evaluated five-replica deployment; extrapolating the same control latency to substantially larger replica sets requires a separate fleet-size study.

Evaluation takeaway. Together, the experiments show that RAVEL’s gains persist across workloads and repeated runs and arise from three complementary mechanisms: immediate prospective accounting keeps uncommitted demand visible,

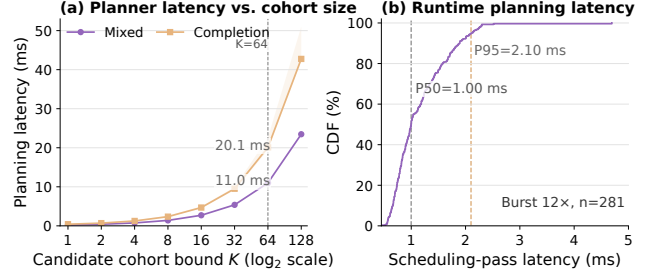


Figure 7: **Planner overhead on the five-replica testbed.** (a) Median planner latency versus candidate cohort bound K ; shaded bands show P5–P95 over 50 repetitions, with $K = 64$ the online bound and $K = 128$ stress-only. (b) Empirical CDF of scheduling-pass latency over 281 actual Burst 12 $\times$ passes.

pre-handoff replanning corrects stale allocations, and engine-local TBT protection preserves the feasibility assumptions used by the Router. Replica-specific quotes provide a decision-useful feasibility signal, while bounded replanning remains millisecond-scale on the evaluated online path.

6 Related Work

LLM serving engines and phase scheduling. Modern LLM serving systems improve execution efficiency through iteration-level batching and KV-cache management [14, 52], scalable and parallel inference [4, 18, 32, 45], optimized attention and speculative execution [6, 28], and structured or adapter-multiplexed serving [37, 57]. A complementary line of work explicitly manages the distinct resource demands of Prefill and Decode. Sarathi-Serve uses Chunked Prefill to reduce Prefill–Decode interference [3], DistServe disaggregates the two phases and provisions them against latency objectives [58], and Splitwise and ShuffleInfer similarly separate or coordinate phase execution to reduce interference [12, 31]. These systems primarily improve how admitted requests execute within the serving engine. RAVEL operates one control layer earlier: it keeps geographic placement revisable while the request remains Router-owned, and relies on engine-local controls only to preserve the service assumptions used by that placement decision.

Distributed and cache-aware LLM routing. Distributed LLM serving systems increasingly choose destinations using runtime load, reusable KV state, and adaptive scheduling signals. Preble jointly considers reusable KV state and computation load [39], while DualMap balances cache affinity against load and performs hotspot-aware rebalancing [53]. Other work explores learned or adaptive request scheduling [8, 19, 55]. A related body of work improves KV-cache reuse, storage, and transfer through cache-centric serving and prefix-aware execution [1, 9, 15, 33, 49–51]. These techniques improve destination selection or reduce the cost of serving a re-

quest at the selected destination. RAVEL addresses a different lifecycle decision. An arriving request contributes prospective demand to its tentative replica immediately, while that destination remains revisable until commitment. Thus, RAVEL does not replace load-, cache-, or locality-aware placement signals; it changes how long a placement may remain tentative and how tentative work is represented to subsequent decisions.

Geo-distributed and network-aware placement. Recent systems explicitly incorporate geographic and network heterogeneity into LLM placement. SkyWalker balances cross-region load while preserving KV-cache locality [48]; GORGON incorporates WAN latency, queueing state, and Pre-fill/cache cost into cross-region decisions [43]; and Solyx uses serving telemetry together with WAN measurements for per-request cross-site placement [5]. Helix considers heterogeneous GPU and network resources in LLM serving [26], while network-aware ML schedulers such as CASSINI coordinate placement with communication conditions [34]. More broadly, private-WAN systems such as SWAN and B4 demonstrate the importance of heterogeneous inter-datacenter paths and traffic constraints [11, 13]. These systems sharpen the decision of *where* work should execute given the state visible to the scheduler. RAVEL makes a second decision first-class: *when should that placement become irreversible?* Its answer is to expose arrived demand to planning immediately while preserving pre-handoff revisability.

SLO- and application-aware LLM scheduling. Prior systems adapt LLM execution and routing to request-specific latency, deadline, or quality objectives. JITServe schedules heterogeneous request classes from imprecise request information [56], QUARTZ targets TTFT SLOs through quantile-aware routing and queueing [24], Andes considers quality-of-experience for streamed generation [22], and AdaServe customizes speculative decoding to per-request SLO requirements [17]. Libra combines request partitioning with SLO-aware scheduling under dynamic load [35], while complementary work studies serving fairness, SLO metrics, and serving-performance characterization [2, 38, 44]. Application-aware systems exploit structure across related LLM calls: Parrot coordinates semantic dependencies across requests [20], Agentix schedules agent programs as first-class execution entities [25], and Ayo optimizes end-to-end LLM applications [42]. Agent frameworks and benchmarks further expose increasingly multi-stage and dependency-rich workloads [23, 27, 47, 59]. RAVEL complements these systems by using request-specific SLO semantics to determine which geographic replicas remain feasible at each decision point. The distinction is lifecycle-level: those feasibility signals guide tentative placement and replanning before handoff, rather than only ranking a single irrevocable destination.

Reassignment, migration, and online commitment. Serving systems can preserve flexibility after an initial assignment through runtime migration, state movement, preemption, or dynamic orchestration [7, 29, 40, 41, 46]. Llumix, for example, dynamically reschedules requests across model instances and uses live migration to improve load balancing and isolation [41]. Such mechanisms retain flexibility after engine-managed execution state has materialized. RAVEL instead preserves flexibility before that point: a request may be re-assigned while Router-owned, but successful engine handoff makes its geographic placement final. Pre-handoff replanning and runtime migration are therefore complementary; the former can correct stale allocations before state movement is necessary, while the latter can address imbalance that arises after execution begins.

Delay scheduling provides another relevant comparison. Waiting-based policies preserve optionality by postponing action [54], while deadline-aware and online schedulers study finite slack, incomplete information, and scalable decision making [10, 16, 21, 30, 36]. RAVEL differs in what remains visible during that optionality window. A Router-owned request may remain uncommitted, but it is already tentatively placed and prospectively accounted. Thus, preserving placement flexibility does not require withholding already-arrived demand from the state used to make subsequent decisions.

7 Conclusion

Geo-distributed LLM capacity is request-dependent: a region that is optional for one request may be indispensable to another. This makes irreversible arrival-time placement vulnerable to avoidable allocation-induced SLO misses as later demand changes the opportunity cost of regional capacity.

Our diagnosis shows that simply delaying commitment is insufficient. At an 80-ms fixed late-binding delay, removing the direct waiting penalty recovers only 0.6 percentage points of SLO attainment, whereas making held requests immediately visible to prospective capacity accounting recovers 15.0 points. The dominant recoverable loss therefore comes from delaying demand visibility together with placement finality.

RAVEL separates these lifecycle decisions. Requests enter capacity planning immediately through tentative placement and prospective accounting, remain revisable while Router-owned, and become final only after successful engine handoff. Replica-specific SLO quotes, bounded pre-handoff replanning, and engine-local protection make this separation practical.

On our physical-WAN vLLM deployment, RAVEL improves request-level SLO attainment across conversational, bursty, and multi-stage workloads while retaining millisecond-scale online planning overhead. More broadly, preserving online flexibility need not require hiding already-arrived work from the state used to make future decisions.

References

- [1] AGARWAL, S., SUNDARESAN, S., MITRA, S., MAHAPATRA, D., GUPTA, A., SHARMA, R., KAPU, N. J., YU, T., AND SAINI, S. Cachecraft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [2] AGRAWAL, A., KEDIA, N., MOHAN, J., PANWAR, A., KWATRA, N., GULAVANI, B. S., RAMJEE, R., AND TUMANOV, A. VIDUR: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems* (2024), vol. 6, pp. 351–366.
- [3] AGRAWAL, A., KEDIA, N., PANWAR, A., MOHAN, J., KWATRA, N., GULAVANI, B., TUMANOV, A., AND RAMJEE, R. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 117–134.
- [4] AMINABADI, R. Y., RAJBHANDARI, S., AWAN, A. A., LI, C., LI, D., ZHENG, E., RUWASE, O., SMITH, S., ZHANG, M., RASLEY, J., AND HE, Y. Deepspeed- inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis* (2022), IEEE, pp. 1–15.
- [5] BERNHARD, A., AND KATLA, N. Solyx AI grid: Hardware-telemetry-aware routing across geographically distributed GPU clusters, 2026. arXiv:2606.15050v1.
- [6] DAO, T. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
- [7] FU, Y., XUE, L., HUANG, Y., BRABETE, A.-O., USTIUGOV, D., PATEL, Y., AND MAI, L. ServerlessLLM: Low-Latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 135–153.
- [8] FU, Y., ZHU, S., SU, R., QIAO, A., STOICA, I., AND ZHANG, H. Efficient llm scheduling by learning to rank. In *Advances in Neural Information Processing Systems 37* (2024), Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 59006–59029.
- [9] GAO, B., HE, Z., SHARMA, P., KANG, Q., JEVDJIC, D., DENG, J., YANG, X., YU, Z., AND ZUO, P. Cost-Efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 111–126.
- [10] GOG, I., SCHWARZKOPF, M., GLEAVE, A., WATSON, R. N. M., AND HAND, S. Firmament: Fast, centralized cluster scheduling at scale. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (Savannah, GA, Nov. 2016), USENIX Association, pp. 99–115.
- [11] HONG, C.-Y., KANDULA, S., MAHAJAN, R., ZHANG, M., GILL, V., NANDURI, M., AND WATTENHOFER, R. Achieving high utilization with software-driven wan. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 15–26.
- [12] HU, C., HUANG, H., XU, L., CHEN, X., XU, J., CHEN, S., FENG, H., WANG, C., WANG, S., BAO, Y., SUN, N., AND SHAN, Y. Inference without interference: Disaggregate llm inference for mixed downstream workloads, 2024.
- [13] JAIN, S., KUMAR, A., MANDAL, S., ONG, J., POUTIEVSKI, L., SINGH, A., VENKATA, S., WANDERER, J., ZHOU, J., ZHU, M., ZOLLA, J., HÖLZLE, U., STUART, S., AND VAHDAT, A. B4: Experience with a globally-deployed software defined wan. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 3–14.
- [14] KWON, W., LI, Z., ZHUANG, S., SHENG, Y., ZHENG, L., YU, C. H., GONZALEZ, J. E., ZHANG, H., AND STOICA, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles* (2023), SOSP ’23, ACM, pp. 611–626.
- [15] LEE, W., LEE, J., SEO, J., AND SIM, J. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 155–172.
- [16] LI, W., HE, X., LIU, Y., LI, K., CHEN, K., GE, Z., GUAN, Z., QI, H., ZHANG, S., AND LIU, G. Flow scheduling with imprecise knowledge. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (Santa Clara, CA, Apr. 2024), USENIX Association, pp. 95–111.
- [17] LI, Z., CHEN, Z., DELACOURT, R., OLIARO, G., WANG, Z., CHEN, Q., LIN, S., YANG, A., ZHANG, Z., CHEN, Z., LAI, Y.-H., CHENG, X., MIAO, X., AND JIA, Z. AdaServe: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding. In *Proceedings of the 21st European Conference on Computer Systems* (2026), EuroSys ’26, ACM, pp. 36–54.
- [18] LI, Z., ZHENG, L., ZHONG, Y., LIU, V., SHENG, Y., JIN, X., HUANG, Y., CHEN, Z., ZHANG, H., GONZALEZ, J. E., AND STOICA, I. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)* (Boston, MA, July 2023), USENIX Association, pp. 663–679.
- [19] LIM, G., ZHAO, W., GODFREY, B., SHAN, J., XU, L., AND XIE, L. Lodestar: An online-learning llm inference router, 2026.
- [20] LIN, C., HAN, Z., ZHANG, C., YANG, Y., YANG, F., CHEN, C., AND QIU, L. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 929–945.
- [21] LIU, C. L., AND LAYLAND, J. W. Scheduling algorithms for multi-programming in a hard-real-time environment. *Journal of the ACM* 20, 1 (1973), 46–61.
- [22] LIU, J., CHUNG, J.-W., WU, Z., LAI, F., LEE, M., AND CHOWDHURY, M. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services, 2024.
- [23] LIU, X., YU, H., ZHANG, H., XU, Y., LEI, X., LAI, H., GU, Y., DING, H., MEN, K., YANG, K., ZHANG, S., DENG, X., ZENG, A., DU, Z., ZHANG, C., SHEN, S., ZHANG, T., SU, Y., SUN, H., HUANG, M., DONG, Y., AND TANG, J. Agentbench: Evaluating llms as agents, 2025.
- [24] LIU, Z., ZHENG, Y., KONG, F., AND ZHAO, Z. QUARTZ: Quantile-aware routing and queueing for TTFT SLOs in LLM serving. In *Findings of the Association for Computational Linguistics: ACL 2026* (San Diego, California, United States, July 2026), Association for Computational Linguistics, pp. 37886–37896.
- [25] LUO, M., SHI, X., CAI, C., ZHANG, T., WONG, J., WANG, Y., WANG, C., HUANG, Y., CHEN, Z., GONZALEZ, J. E., AND STOICA, I. Agentix: An efficient serving engine for LLM agents as general programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)* (Renton, WA, May 2026), USENIX Association, pp. 2443–2459.
- [26] MEI, Y., ZHUANG, Y., MIAO, X., YANG, J., JIA, Z., AND VINAYAK, R. Helix: Serving large language models over heterogeneous gpus and network via max-flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (2025), ACM, pp. 586–602.
- [27] MIALON, G., FOURRIER, C., SWIFT, C., WOLF, T., LECUN, Y., AND SCIALOM, T. Gaia: a benchmark for general ai assistants, 2023.
- [28] MIAO, X., OLIARO, G., ZHANG, Z., CHENG, X., WANG, Z., ZHANG, Z., WONG, R. Y. Y., ZHU, A., YANG, L., SHI, X., SHI, C., CHEN, Z., ARFEEN, D., ABHYANKAR, R., AND JIA, Z. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (2024), ACM, pp. 932–949.

- [29] MIAO, X., SHI, C., DUAN, J., XI, X., LIN, D., CUI, B., AND JIA, Z. Spotservice: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2024), ACM, pp. 1112–1127.
- [30] OUSTERHOUT, K., WENDELL, P., ZAHARIA, M., AND STOICA, I. Sparrow: Distributed, low latency scheduling. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (2013), SOSP '13, ACM, pp. 69–84.
- [31] PATEL, P., CHOUKSE, E., ZHANG, C., SHAH, A., ÍÑIGO GOIRI, MALEKI, S., AND BIANCHINI, R. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)* (2024), IEEE, pp. 118–132.
- [32] POPE, R., DOUGLAS, S., CHOWDERY, A., DEVLIN, J., BRADBURY, J., LEVSKAYA, A., HEEK, J., XIAO, K., AGRAWAL, S., AND DEAN, J. Efficiently scaling transformer inference, 2022.
- [33] QIN, R., LI, Z., HE, W., CUI, J., REN, F., ZHANG, M., WU, Y., ZHENG, W., AND XU, X. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)* (Santa Clara, CA, Feb. 2025), USENIX Association, pp. 155–170.
- [34] RAJASEKARAN, S., GHOBADI, M., AND AKELLA, A. CASSINI: Network-Aware job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)* (Santa Clara, CA, Apr. 2024), USENIX Association, pp. 1403–1420.
- [35] RUAN, C., CHEN, Y., TIAN, D., SHI, Y., WU, Y., LI, J., AND LI, C. Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)* (Renton, WA, May 2026), USENIX Association, pp. 1243–1258.
- [36] SCHWARZKOPF, M., KONWINSKI, A., ABD-EL-MALEK, M., AND WILKES, J. Omega: Flexible, scalable schedulers for large compute clusters. In *Proceedings of the 8th ACM European Conference on Computer Systems* (2013), ACM, pp. 351–364.
- [37] SHENG, Y., CAO, S., LI, D., HOOPER, C., LEE, N., YANG, S., CHOU, C., ZHU, B., ZHENG, L., KEUTZER, K., GONZALEZ, J. E., AND STOICA, I. S-LoRA: Serving thousands of concurrent LoRA adapters. In *Proceedings of Machine Learning and Systems* (2024), vol. 6.
- [38] SHENG, Y., CAO, S., LI, D., ZHU, B., LI, Z., ZHUO, D., GONZALEZ, J. E., AND STOICA, I. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 965–988.
- [39] SRIVATSA, V., HE, Z., ABHYANKAR, R., LI, D., AND ZHANG, Y. Preble: Efficient distributed prompt scheduling for LLM serving. In *The Thirteenth International Conference on Learning Representations* (2025).
- [40] STRATI, F., MCALLISTER, S., PHANISHAYEE, A., TARNAWSKI, J., AND KLIMOVIC, A. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving, 2024.
- [41] SUN, B., HUANG, Z., ZHAO, H., XIAO, W., ZHANG, X., LI, Y., AND LIN, W. Lumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 173–191.
- [42] TAN, X., JIANG, Y., YANG, Y., AND XU, H. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2025), ACM, pp. 1302–1316.
- [43] TONIOLO, A. R., THORSTENSON, R., AND DINESH, A. GORGO: Online tuning for cross-region network-aware LLM serving, 2026. arXiv:2602.11688v3.
- [44] WANG, Z., LI, S., ZHOU, Y., LI, X., ZHANG, Z., CAM-TU, N., GU, R., TIAN, C., CHEN, G., AND ZHONG, S. Revisiting service level objectives and system level metrics in large language model serving, 2024.
- [45] WU, B., LIU, S., ZHONG, Y., SUN, P., LIU, X., AND JIN, X. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (2024), ACM, pp. 640–654.
- [46] WU, B., ZHU, R., ZHANG, Z., SUN, P., LIU, X., AND JIN, X. dLoRA: Dynamically orchestrating requests and adapters for LoRA LLM serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 911–927.
- [47] WU, Q., BANSAL, G., ZHANG, J., WU, Y., LI, B., ZHU, E., JIANG, L., ZHANG, X., ZHANG, S., LIU, J., AWADALLAH, A. H., WHITE, R. W., BURGER, D., AND WANG, C. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.
- [48] XIA, T., MAO, Z., KERNEY, J., JACKSON, E. J., LI, Z., XING, J., SHENKER, S., AND STOICA, I. SkyWalker: A locality-aware cross-region load balancer for LLM inference. In *Proceedings of the 21st European Conference on Computer Systems* (2026), EuroSys '26, ACM, pp. 1349–1364.
- [49] YANG, J., HOU, B., WEI, W., BAO, Y., AND CHANG, S. Kvlink: Accelerating large language models via efficient kv cache reuse. In *Advances in Neural Information Processing Systems 38* (2025), Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 148312–148339.
- [50] YAO, J., LI, H., LIU, Y., RAY, S., CHENG, Y., ZHANG, Q., DU, K., LU, S., AND JIANG, J. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems* (2025), ACM, pp. 94–109.
- [51] YE, L., TAO, Z., HUANG, Y., AND LI, Y. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), Association for Computational Linguistics, pp. 11608–11620.
- [52] YU, G.-I., JEONG, J. S., KIM, G.-W., KIM, S., AND CHUN, B.-G. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)* (Carlsbad, CA, July 2022), USENIX Association, pp. 521–538.
- [53] YUAN, Y., ZUO, P., WANG, B., CHEN, Z., TAN, Z., AND YU, Z. DualMap: Enabling both cache affinity and load balancing for distributed LLM serving. In *The Fourteenth International Conference on Learning Representations* (2026).
- [54] ZAHARIA, M., BORTHAKUR, D., SARMA, J. S., ELMELEEGY, K., SHENKER, S., AND STOICA, I. Delay scheduling: A simple technique for achieving locality and fairness in cluster scheduling. In *Proceedings of the 5th European Conference on Computer Systems* (2010), EuroSys '10, ACM, pp. 265–278.
- [55] ZHANG, D., HAN, J., ZHANG, K., WEI, X., SHEN, S., FANG, C., YU, W., ZHOU, J., AND CHEN, R. Simple is better: Multiplication may be all you need for llm request scheduling, 2026.
- [56] ZHANG, W., WU, Z., MU, Y., NING, R., LIU, B., SARDA, N., LEE, M., AND LAI, F. JITServe: SLO-aware LLM serving with imprecise request information. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)* (Renton, WA, May 2026), USENIX Association, pp. 825–848.

- [57] ZHENG, L., YIN, L., XIE, Z., SUN, C., HUANG, J., YU, C., CAO, S., KOZYRAKIS, C., STOICA, I., GONZALEZ, J., BARRETT, C., AND SHENG, Y. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37* (2024), Neural Information Processing Systems Foundation, Inc. (NeurIPS), pp. 62557–62583.
- [58] ZHONG, Y., LIU, S., CHEN, J., HU, J., ZHU, Y., LIU, X., JIN, X., AND ZHANG, H. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 193–210.
- [59] ZHOU, S., XU, F. F., ZHU, H., ZHOU, X., LO, R., SRIDHAR, A., CHENG, X., OU, T., BISK, Y., FRIED, D., ALON, U., AND NEUBIG, G. Webarena: A realistic web environment for building autonomous agents, 2024.

A Detailed Design and Operational Semantics

A.1 Common Request State and Notation

RAVEL separates *placement ownership* from engine execution state. Before successful engine handoff, a request is Router-owned: it has one tentative replica, contributes prospective demand to that replica, and may be replanned subject to the mobility rules below. Successful engine handoff transfers ownership to the serving engine and makes the geographic placement final. Engine-side execution may subsequently move through Prefill, Decode, completion, or failure states, but these events do not reopen geographic placement.

For request i , a_i denotes its Router arrival time, o_i its client region, and $r_i(t)$ its tentative replica while Router-owned. The version counter v_i protects versioned ownership and accounting transfers, while h_i^{att} identifies the current engine attempt. Attempt identifiers isolate asynchronous completion, cancellation, and cleanup events across retries.

The Router-side transport path distinguishes `WAITING_MOVABLE` from the transient `COMMITTING` state around engine handoff. A dispatch claim enters `COMMITTING`; this is an ownership-transition attempt rather than a second placement decision. If target registration or pre-handoff posting setup fails, Router ownership is restored or cleaned without reporting a committed placement. Placement becomes final only after the target accepts registration and the engine-facing posting task is created and tracked successfully. RAVEL does not perform ordinary geographic rebinds after this boundary.

These operational rules refine the three invariant classes from Section 3.1. Specifically, RAVEL ensures that (i) each Router-owned request has exactly one tentative owner; (ii) its prospective demand is represented exactly once and follows reassignment; (iii) only Router-owned requests may change geographic placement; (iv) replanning is causal and bounded in observation time and cohort size; and (v) only successful engine handoff commits placement. Attempt identifiers additionally isolate asynchronous callbacks across retries: a callback from attempt h cannot modify state belonging to a newer attempt $h' > h$.

A.2 Replica SLO Quote Construction

A placement decision consumes the causal quote

$$\mathcal{Q}_{i,r}(t) = \left(\Phi_{i,r}(t), \hat{T}_{i,r}^{\text{obj}}(t), \mathbf{z}_{i,r}(t) \right), \quad (7)$$

where $\Phi_{i,r}(t)$ is predicted SLO feasibility, $\hat{T}_{i,r}^{\text{obj}}(t)$ is the class-specific ordering objective, and $\mathbf{z}_{i,r}(t)$ contains secondary capacity-pressure and stability signals. Quotes use only information available at decision time and are not formal SLO guarantees. Their inputs include request-visible SLO metadata, directed WAN measurements, calibrated per-cluster service

profiles, observed engine state, Router-owned queue state, and tentative assignments already selected in the current planning pass. The evaluated request’s realized output length is never an input to its quote.

Calibrated service curves. For cluster c and active Decode concurrency n , the service profile stores an affine Prefill model

$$P_{c,n}(p) = b_{c,n} + a_{c,n}p \quad (8)$$

and a measured post-first-token Decode time per token $d_c(n)$. Profiles are calibrated independently for each serving cluster and carry an engine fingerprint. At runtime, RAVEL linearly interpolates between measured concurrency points and clamps values outside the calibrated range to the nearest endpoint. Because $P_{c,n}$ is measured while n Decode sequences are active, RAVEL does not add a separate ad-hoc Decode-interference penalty to the Prefill estimate.

Queue-ahead Prefill expansion. Let p_i be the full prompt-token charge of request i . Without engine-certified cached-token residency, shadow prefix locality is used only as a late placement tie-breaker and does not reduce p_i .

For candidate replica r , let E_r denote observed engine-pending Prefill tokens, $B_{i,r}$ the prompt tokens in Router-owned requests ordered no later than i , N_r^{eng} the number of engine-pending requests, and $N_{i,r}^q$ the corresponding number of Router-owned queue-ahead requests. With n_r currently running Decode sequences, RAVEL estimates

$$\hat{S}_{i,r}^{\text{pre}} = a_{c(r),n_r}(E_r + B_{i,r} + p_i) + b_{c(r),n_r}(N_r^{\text{eng}} + N_{i,r}^q + 1). \quad (9)$$

During cohort replanning, $B_{i,r}$ and $N_{i,r}^q$ additionally include tentative assignments selected earlier in the same planning pass. Thus, later candidates observe prospective work induced by earlier tentative choices rather than a stale snapshot.

Using the notation of Section 3.2, the TTFT estimate is

$$\widehat{\text{TTFT}}_{i,r}(t) = (t - a_i) + \rho_{o_i,c(r)} + \hat{S}_{i,r}^{\text{pre}}, \quad (10)$$

where $\rho_{o_i,c(r)}$ is the directed client-to-cluster WAN delay. The elapsed term explicitly charges Router observation time against the request’s remaining SLO budget.

Prospective Decode occupancy. Let O_r be the union of request identifiers visible in the Router-owned queue, engine-pending set, and engine-running set at replica r . RAVEL forms

$$m_{i,r} = \max(1, |O_r \cup \{i\}|, N_r^q + N_r^{\text{eng}} + N_r^{\text{run}} + \mathbf{1}[i \notin O_r]), \quad (11)$$

and queries the calibrated Decode curve at $m_{i,r}$, clamping at the measured profile endpoint. The set term prevents the same request from being charged twice when lifecycle views overlap.

LATENCY feasibility. For a LATENCY request with TTFT and TBT limits D_i^{TTFT} and D_i^{TBT} ,

$$\Phi_{i,r}^{\text{lat}}(t) = \mathbf{1} \left[\widehat{\text{TTFT}}_{i,r}(t) \leq D_i^{\text{TTFT}} \wedge d_{c(r)}(m_{i,r}) \leq D_i^{\text{TBT}} \right]. \quad (12)$$

The two constraints are evaluated independently: a low predicted TTFT cannot compensate for a predicted TBT violation.

Completion feasibility and output demand. For completion-oriented requests, RAVEL distinguishes request-visible expected output demand L_i^{exp} from conservative risk demand L_i^{risk} . The corresponding completion estimates are

$$\widehat{T}_{i,r}^{\text{exp}}(t) = \widehat{\text{TTFT}}_{i,r}(t) + \max(0, L_i^{\text{exp}} - 1) d_{c(r)}(m_{i,r}), \quad (13)$$

$$\widehat{T}_{i,r}^{\text{risk}}(t) = \widehat{\text{TTFT}}_{i,r}(t) + \max(0, L_i^{\text{risk}} - 1) d_{c(r)}(m_{i,r}). \quad (14)$$

Completion feasibility uses

$$\Phi_{i,r}^{\text{comp}}(t) = \mathbf{1} \left[\widehat{T}_{i,r}^{\text{risk}}(t) \leq D_i^{\text{comp}} \right], \quad (15)$$

while $\widehat{T}_{i,r}^{\text{exp}}$ represents expected shared service pressure.

Ordinary feasibility always uses the conservative L_i^{risk} estimate. Under the completion-pressure path in Section A.6, a held-out semantic profile may replace the placement-time estimate of L_i^{exp} using only request-visible workflow metadata. It does not replace L_i^{risk} , reveal future arrivals, or use the evaluated request’s realized output length.

Quote procedure. For completeness, a replica quote performs the following operations:

QUOTE_REPLICA(i, r, t): (1) read calibrated service curves and currently observed engine state; (2) charge full prompt work and Router-owned work ahead of i ; (3) add tentative work selected earlier in the current planning pass; (4) compute Eq. 10; (5) form the request-ID-deduplicated prospective Decode count in Eq. 11; (6) evaluate the applicable LATENCY or completion feasibility predicate; and (7) return feasibility, the class-specific objective, and secondary pressure/locality signals.

A.3 Prospective Accounting Lifecycle

RAVEL uses prospective state for two purposes: to make newly arrived work visible before commitment and to ensure that sequential decisions within one planning pass account for tentative work selected earlier in that pass.

Queue and service debt. The queue-ahead terms in Eq. 9 derive from Router-owned and engine-visible state. The RAVEL planner additionally maintains virtual ledgers so that

newly tentative work affects subsequent placement decisions before engine lifecycle feedback arrives.

For a LATENCY request,

$$u_{i,r} = \frac{d_{c(r)}(1)}{D_i^{\text{TBT}}}, \quad A_r = \sum_{j: \text{latency owner}(j)=r} u_{j,r}. \quad (16)$$

Both numerator and denominator are seconds, so $u_{i,r}$ is dimensionless and represents one-token-period service demand. This ledger is a placement-balancing signal rather than an estimate of the instantaneous vLLM Decode queue.

For requests with active service accounting, define

$$s_{i,r}^{\text{own}} = \widehat{s}_{i,r}^{\text{own,pre}} + \widehat{s}_{i,r}^{\text{own,dec}}, \quad L_r = \sum_{j: \text{service owner}(j)=r} s_{j,r}^{\text{own}}. \quad (17)$$

The Prefill component contains the request’s own profiled service, including the profile intercept, rather than queue-ahead debt. The Decode component uses the request-visible expected service represented by the corresponding quote. Thus L_r is measured in seconds of profiled service.

Exactly-once registration and transfer. Each persistent virtual charge is associated with an explicit request owner. Registration is idempotent: if the corresponding owner field already exists, the request is not added again.

For any transferable prospective ledger Z , reassignment from r to r' performs

$$Z_r \leftarrow Z_r - z_{i,r}, \quad Z_{r'} \leftarrow Z_{r'} + z_{i,r'}, \quad (18)$$

where the destination charge is recomputed for r' . The remove–recompute–install sequence executes atomically on the serialized Router control path.

During cohort replanning, RAVEL first removes mobile candidates’ old transferable charges from the fixed planning base. Candidates are then processed in planning order; after each tentative target is selected, its new prompt, count, and service charges are added immediately to pass-local state. Persistent ownership is updated only after the corresponding version and attempt checks succeed.

For completion-oriented accounting, the active service charge is released by the matching terminal callback. The LATENCY token-period ledger remains a placement-balancing signal; actual Decode feasibility uses the request-ID-deduplicated prospective count in Eq. 11.

Operationally,

TRANSFER_OR_REFRESH($i, r \rightarrow r'$): (1) verify that i is still Router-owned and its version/attempt is current; (2) remove its old transferable charges from the planning base; (3) recompute $Q_{i,r'}$ against fixed and pass-local state; (4) update pass-local ledgers immediately if r' is selected; (5) atomically replace persistent owner and charge fields; and (6) record the updated tentative quote. A failed version or attempt check leaves the current owner unchanged.

Table 3: Prospective-accounting lifecycle.

Event	Router owner	Prospective	Active
Arrival	Install	Install	–
Reassignment	Move	Recompute + move	–
Rejected handoff	Restore	Restore	–
Successful handoff	Remove	Remove	Install
Terminal event	–	–	Release

A.4 Bounded Pre-Handoff Replanning

RAVEL replans from causally observed state rather than polling for an unbounded future window. Scheduling operations are serialized by one asynchronous scheduler lock, and multiple requests for a planning pass may be coalesced.

Replanning triggers and eligibility. A planning pass may be requested by: (i) a new arrival, (ii) mobility-hold expiration, (iii) a relevant engine-state update, (iv) request termination, (v) rejected handoff, or (vi) a completion-pressure state change. Only Router-owned requests remain eligible for ordinary geographic reassignment. A candidate is excluded once it has crossed the handoff boundary, has no alternative SLO-feasible target, or has exhausted the configured rebind limit.

Slack-bounded observation. Let δ_i be the most recently observed interarrival gap and $\mathcal{D}_i$ the set of positive RTTs from request i 's origin to remote serving clusters. The causal burst predicate is

$$\text{Burst}_i = \mathbf{1}[\mathcal{D}_i \neq \emptyset \wedge \delta_i \leq \max \mathcal{D}_i]. \quad (19)$$

Let Δ_i denote the applicable planning budget: D_i^{TTFT} for LATENCY requests and D_i^{comp} for completion-oriented requests. Arrival-time planning slack is

$$\sigma_i^{\text{arr}} = \max\left\{0, \Delta_i - \min_r \hat{T}_{i,r}^{\text{obj}}(a_i)\right\}. \quad (20)$$

Let $\rho_i^{\min} = \min \mathcal{D}_i$ when $\mathcal{D}_i \neq \emptyset$. Hold eligibility requires (i) a non-LATENCY request, (ii) $\text{Burst}_i = 1$, and (iii) at least two SLO-feasible serving regions. The mobility hold is

$$h_i = \begin{cases} \min\{\rho_i^{\min}, \sigma_i^{\text{arr}}\}, & \text{if } i \text{ is hold-eligible,} \\ 0, & \text{otherwise.} \end{cases} \quad (21)$$

LATENCY requests are not deliberately held. The hold changes dispatch eligibility rather than accounting: a held request remains tentatively placed, prospectively accounted, and revisable throughout the observation interval.

Movable cohort. Movable requests are ordered lexicographically by

$$\kappa_i = (a_i + \Delta_i, a_i, id_i), \quad (22)$$

which gives a deterministic deadline-first order. The ordinary movable-candidate bound is K ; under completion pressure, Section A.6, the effective bound may expand to aggregate sequence capacity across replicas.

Fixed and pass-local context. A planning pass freezes non-candidate engine and Router context. Fixed Router-owned requests that precede candidate i under Eq. 22 contribute their prompt work and request intercept to i 's quote. After a target is selected for a movable candidate, its prompt work, request count, service charge, and applicable TBT charge are immediately added to pass-local state before the next candidate is evaluated.

The full planning procedure is:

REPLAN: (1) collect the bounded movable cohort and sort it by Eq. 22; (2) freeze current noncandidate engine and Router state; (3) remove candidates' old transferable charges from the fixed planning base; (4) for each candidate, quote every legal replica, choose the minimum lexicographic placement key, and update pass-local state immediately; (5) apply only still-current assignments, respecting attempt/version and rebind guards; and (6) dispatch newly eligible work through the ordinary dispatch frontier.

Let N be the movable cohort size, R the number of replicas, and F the number of fixed Router-owned requests scanned to construct queue-ahead prefixes. The planning cost is

$$O(N \log N + F \log F + NR + F), \quad (23)$$

plus constant-size service-profile interpolation. The NR term is the request–replica quote-comparison cost; with $N \leq K_{\text{eff}}$, it corresponds to the $O(K_{\text{eff}}R)$ comparison bound stated in Section 3.4. The remaining terms account for deterministic cohort ordering and fixed-context construction. The bound is independent of generated token count; RAVEL does not require a Router scheduling event for every generated token.

A.5 Deterministic Placement Key

RAVEL avoids combining heterogeneous SLO and load quantities in a fitted weighted sum. The planner compares candidate replica r for request i using the lexicographic key

$$K_{i,r} = (\neg\Phi_{i,r}, O_{i,r}, \ell_{i,r}^{\text{obj}}, \ell_{i,r}^{\text{TTFT}}, A_r + u_{i,r}, L_r + s_{i,r}^{\text{own}}, \hat{T}_{i,r}^{\text{obj}}/D_i, M_{i,r}, \text{moved}_{i,r}, id_r), \quad (24)$$

with lower values preferred. Terms inapplicable to a request class use their neutral value rather than inducing a separate class-specific placement algorithm.

The exact order can be interpreted in three groups. The *SLO-safety* prefix first excludes predicted-infeasible targets

and applies the pending-capacity overflow guard $O_{i,r}$ together with normalized SLO lateness. *Capacity-pressure* terms then compare prospective LATENCY/TBT and completion-service pressure. Finally, *efficiency and stability* use the predicted objective, marginal-service cost, movement, and deterministic replica identity. Lower-priority terms cannot override an earlier safety decision.

The normalized lateness terms are

$$\ell_{i,r}^{\text{obj}} = \frac{\max(0, \hat{T}_{i,r}^{\text{obj}} - D_i)}{\max(D_i, \epsilon)}, \quad (25)$$

$$\ell_{i,r}^{\text{TTFT}} = \frac{\max(0, \widehat{\text{TTFT}}_{i,r} - D_i^{\text{TTFT}})}{\max(D_i^{\text{TTFT}}, \epsilon)}. \quad (26)$$

$O_{i,r}$ is the pending-capacity overflow cost when that guard is applicable.

The convex marginal-service term is

$$M_{i,r} = \frac{(W_r + s_{i,r}^{\text{own}})^2 - W_r^2}{D_i^2}, \quad (27)$$

where W_r is the profiled service already represented at replica r in the current planning state. Equation 27 is the discrete increment of a quadratic congestion potential rather than a learned weight.

Initial single-request placement uses the same SLO-safety and capacity ordering. Shadow prefix locality, when consulted, is restricted to a late tie-break path and never precedes predicted feasibility or normalized SLO violation.

During cohort replanning, the stronger invariant is pass-local visibility: after each tentative assignment, the planner updates prospective state before quoting the next candidate. Later candidates therefore observe demand induced by earlier choices in the same pass rather than a stale pre-pass snapshot.

A.6 Completion-Pressure Adaptation

The completion-pressure path is enabled by request-visible workflow semantics, not by a workload or dataset name. A request is a workflow-completion request when it is COLLECTIVE and contains more than one stage. RAVEL tracks the number of such arrivals, their first and latest arrival times, the minimum observed completion SLO, whether nonworkflow traffic has appeared, and the amount of outstanding workflow work.

Let C_{seq} be aggregate sequence capacity across replicas and D_{min} the minimum completion SLO among observed workflow requests. After at least N_{min} samples, RAVEL computes

$$q_{\text{obs}} = \frac{N_{\text{arr}} - 1}{t_{\text{last}} - t_{\text{first}}}, \quad q_{\text{act}} = \frac{C_{\text{seq}}}{\max(1 \text{ s}, 0.5 D_{\text{min}})}, \quad (28)$$

where

$$N_{\text{min}} = \max\left(4, \min\left(16, \left\lfloor \frac{C_{\text{seq, replica}}}{4} \right\rfloor\right)\right). \quad (29)$$

Define $\text{WorkflowOnly}(t)$ to indicate that workflow-completion traffic has been observed and no nonworkflow request has yet appeared. The effective pressure path is active when

$$\text{CompletionPressure}(t) = \mathbf{1} \left[\begin{array}{l} N_{\text{arr}} \geq N_{\text{min}} \\ \wedge \text{ObservedPressure}(t) \\ \wedge N_{\text{outstanding}} > 0 \\ \wedge \text{WorkflowOnly}(t) \end{array} \right], \quad (30)$$

where ObservedPressure becomes true once $q_{\text{obs}} \geq q_{\text{act}}$. This observed-pressure bit is causal and sticky within the run. The effective path becomes inactive when its eligibility conditions no longer hold and is cleared after outstanding work drains to zero.

Completion pressure changes only two planner inputs:

1. the movable-candidate ceiling expands from the ordinary bound K to aggregate deployment sequence capacity; and
2. when a held-out semantic profile covers the request context, placement-time L_i^{exp} uses the profile's expected output estimate.

It does not change L_i^{risk} , the feasibility predicate, the placement algorithm, the dispatch path, or the commitment boundary.

Semantic output profile. The held-out profile indexes $(\text{stage_id}, \text{stage_num})$ and stores an expected output estimate for each covered context. Lookup follows

$$\begin{aligned} (\text{stage_id}, \text{stage_num}) &\rightarrow \text{stage_id aggregate} \\ &\rightarrow \text{declared routing hint.} \end{aligned} \quad (31)$$

Neither future arrivals nor the current request's realized output length enters the lookup. When an active completion request terminates, RAVEL releases its active service charge and requests another serialized planning pass.

A.7 Shallow Reversible Dispatch Frontier

The RAVEL dispatch frontier limits how much Router-visible prompt work can cross toward engine state at once. Let

$$H = \max_{c \neq c'} \rho_{c,c'} \quad (32)$$

be the largest configured directed inter-cluster control RTT, and let B_{iter} be the engine's configured `max_num_batched_tokens`. For replica r , let $a_{c(r),0}$ be the calibrated fastest Prefill seconds per token. RAVEL sets

$$F_r = B_{\text{iter}} + \left\lceil \frac{H}{a_{c(r),0}} \right\rceil. \quad (33)$$

Thus, the frontier covers approximately one complete engine scheduling quantum plus the prompt work that the fastest calibrated Prefill path could consume during one worst-case control RTT.

At dispatch time, if E_r is observed engine-pending prompt work, frontier headroom is

$$G_r = \max(0, F_r - E_r). \quad (34)$$

The effective dispatch budget is the minimum of G_r and the replica's ordinary engine budget. A Router-owned request may cross only if its full conservative prompt charge fits the budget. Eligible requests are ordered by applicable absolute SLO deadline, then arrival time and request identifier.

In our five-replica deployment, the maximum directed RTT is $H = 37.9$ ms. We use $B_{\text{iter}} = 16384$ tokens, $\text{max_num_seqs}=64$, and 16-token blocks. The second term of Eq. 33 is deployment-specific and is recomputed from the calibrated service profile.

The frontier is not a protected queue for predicted SLO misses. If a dispatch-eligible request fits the capacity budget, it proceeds; otherwise it remains Router-owned, prospectively accounted, and eligible for replanning. Rejected registration follows the handoff rollback semantics in Section A.9.

A.8 Engine-Local TBT Protection

RAVEL's engine adapter protects the local execution assumptions used by LATENCY quotes. The adapter is installed only when the target vLLM exposes the expected scheduler hooks; it fails fast if `Scheduler._get_priority` or `Scheduler._schedule_chunked_prefill` is unavailable rather than silently running an unvalidated compatibility path.

Let

$$T_r = \min_{i \in \mathcal{D}_{\text{lat}}^{\text{TBT}}} D_i^{\text{TBT}} \quad (35)$$

be the tightest active TBT requirement among LATENCY Decode requests at replica r . From the calibrated profile, the adapter forms the conservative envelope

$$a_r^{\text{guard}} = \max_n a_{c(r),n}, \quad b_r^{\text{guard}} = \max_n b_{c(r),n}. \quad (36)$$

For block size g and ordinary token limit B_{max} , it computes

$$q_{\text{raw}} = \left\lfloor \frac{\max(0, T_r - b_r^{\text{guard}})}{a_r^{\text{guard}}} \right\rfloor, \quad (37)$$

$$q_{\text{blk}} = \min \left(B_{\text{max}}, \max \left(g, g \left\lfloor \frac{q_{\text{raw}}}{g} \right\rfloor \right) \right). \quad (38)$$

If n_r^{dec} Decode sequences are active, each contributes one next-token Decode slot to the next scheduling iteration. The temporary block-aligned batch budget is therefore

$$B_r^{\text{guard}} = g \left\lceil \frac{n_r^{\text{dec}} + q_{\text{blk}}}{g} \right\rceil. \quad (39)$$

The adapter applies

$$B_{\text{iter}} \leftarrow \min(B_{\text{max}}, B_r^{\text{guard}}) \quad (40)$$

for that Chunked-Prefill scheduling call only and restores B_{max} afterward.

If no active Decode request carries a LATENCY TBT requirement, the adapter uses the ordinary vLLM scheduling path. If even one minimum-size block exceeds the profiled TBT budget, the adapter must still expose a legal vLLM block and therefore cannot claim a TBT guarantee in that physical regime. The Router feasibility test is intended to avoid such placements, and any realized violation remains in the evaluation denominator.

Phase-aware priority metadata carries a remaining deadline budget rather than a Router monotonic-clock timestamp. The engine reconstructs its local deadline using its own arrival clock, avoiding an implicit cross-host monotonic-clock synchronization assumption. Non-RAVEL priorities occupy a separate range, so the adapter remains inert for ordinary vLLM work.

A.9 Commitment and Engine-Handoff State Machine

RAVEL defines commitment as a successful ownership transition rather than a timer expiration, queue removal, or registration attempt. A request may be tentatively placed, held, replanned, and made dispatch-eligible while remaining Router-owned.

The engine-facing handoff performs four operations: (1) freeze the latest Router quote for observability; (2) ask the tentative target to register the request; (3) create the asynchronous engine-facing posting task; and (4) install that task in the attempt-indexed task table. Quote freezing is diagnostic and does not commit placement. Successful registration together with successful creation and tracking of the posting task is the sole commitment event.

Before successful handoff, failure may restore Router-owned state or clean a partial attempt without reporting a committed placement. After successful handoff, failures belong to the committed attempt and do not create another geographic-placement opportunity.

Task cleanup is attempt-aware. A deletion or terminal callback carrying attempt identifier h is ignored when the stored task belongs to a different active attempt. This prevents delayed callbacks from releasing ownership or accounting state associated with a newer attempt.

Before successful handoff, RAVEL changes only Router/transport metadata and prospective accounting; engine-managed execution state has not yet been materialized in the form that would require runtime KV-state migration. After handoff, pending and running engine state remains visible to subsequent quotes, but the committed request itself is no longer an ordinary geographic replanning candidate.

Table 4: Engine-handoff state transitions.

Phase	Event	Action	Placement final?
Router-owned	Dispatch claim	Enter <code>COMMITTING</code> and attempt target registration.	No
<code>COMMITTING</code>	Registration rejected	Restore Router ownership and prospective accounting.	No
<code>COMMITTING</code>	Posting setup fails	Abort partial registration and clean attempt-specific state.	No
<code>COMMITTING</code>	Registration and posting-task tracking succeed	Transfer ownership to the serving engine.	Yes
Engine-owned	Completion, failure, or cancellation	Perform attempt-specific terminal cleanup; do not reopen placement.	Yes
Any	Stale callback	Ignore the callback when its attempt identifier is not current.	Unchanged

Table 5: Mechanism variants used in the Burst $12\times$ ablation.

Variant	Prospective accounting	Pre-handoff reassignment	Local TBT protection
RAVEL	Yes	Yes	Yes
NoAccounting	No	Yes	Yes
FixedPlacement	Yes	No	Yes
NoTBTGuard	Yes	Yes	No

B Mechanism-Ablation and Rescue Analysis

This appendix defines the mechanism variants used in Section 5.4 and the conservative trace-level criterion behind the reported pre-handoff rescue statistic. All mechanism results and the rescue decomposition use the same 500-request Burst $12\times$ execution; the decomposition is not a separate experimental campaign.

B.1 Mechanism-Ablation Definitions

The ablations remove one control mechanism at a time while retaining the remaining RAVEL lifecycle wherever possible. Table 5 summarizes the comparison.

NoAccounting. This variant retains replica-specific SLO quotes, the pre-handoff replanning path, the ownership/commitment boundary, and engine-local TBT protection, but suppresses the normal prospective representation of Router-owned tentative assignments in the capacity state consumed by subsequent placement decisions. Live engine pending and running state remains visible. The variant therefore isolates the value of making uncommitted demand visible before handoff.

FixedPlacement. This variant performs RAVEL’s initial SLO-aware placement and retains prospective accounting, dispatch/handoff semantics, and engine-local TBT protection, but disables ordinary pre-handoff reassignment after the initial choice. It isolates the value of keeping Router-owned

placement revisable while holding tentative-demand visibility fixed.

NoTBTGuard. This variant retains global quotes, prospective accounting, placement, and pre-handoff replanning, but disables the engine-local Prefill control and priority behavior used to protect active LATENCY Decode requests. It isolates the local mechanism that preserves the TBT assumptions used by replica-level feasibility decisions.

B.2 Confirmed Pre-Handoff Rescue Criterion

The trace diagnostic asks whether a miss under a frozen pre-handoff placement can be conservatively attributed to an allocation that the same planning episode later repairs. We call the affected request the *victim* and an earlier-arrived request that vacates the region obtained by the victim in the same plan the *blocker*.

A frozen-placement miss is classified as a *confirmed pre-handoff rescue* only when all of the following conditions hold:

1. Under the incumbent allocation frozen at the same pre-handoff planning episode, the victim is predicted infeasible.
2. After the cohort reassignment produced by that episode, the victim is predicted feasible.
3. The victim subsequently satisfies its realized request-level SLO.
4. At least one blocker arrived earlier, occupied the region that the victim obtains, and is moved away from that region by the same plan.

The criterion intentionally requires both a decision-level change in predicted feasibility and a successful realized outcome; cases that cannot establish all four conditions remain unresolved.

Feasible-set reconstruction. The diagnostic must distinguish intrinsic regional infeasibility from congestion created by the very Router cohort being analyzed. For each relevant pre-handoff snapshot, we therefore retain materialized engine load but remove the current Router cohort from prospective placement state. We then test the affected request individually against each candidate region using the same causal quote machinery. This reconstruction yields the diagnostic feasible-region set without treating blocker-induced prospective congestion as an intrinsic property of the region. No future arrivals or realized output lengths are introduced into the quote.

Interpretation boundary. Under this conservative criterion, at least 41.25% of misses under frozen pre-handoff placement are confirmed redirect-rescuable. The remaining at most 58.75% are *unresolved* by this diagnostic; they should not be interpreted as proven capacity shortages or irrecoverable failures. The statistic decomposes the same Burst 12 $\times$ mechanism run used for the ablation rather than supplying an independent experimental sample. It provides trace-level evidence for the mechanism behind the FixedPlacement gap, not a counterfactual guarantee that every classified miss would be repaired under arbitrary execution.

Trace collection. The diagnostic records the planning episode, incumbent and revised ownership, predicted feasibility before and after reassignment, and subsequent realized SLO outcome. These records are buffered during execution and persisted at the end of the run so that instrumentation does not introduce synchronous logging work into the scheduling path.

C Repeated-Run Robustness

This appendix reports a five-run paired robustness study at 16 $\times$ load. The repeated-run statistics are reported separately from single-run headline results because they summarize different experimental repetitions.

C.1 Result Context and Comparator Definitions

For DeepResearch at 16 $\times$ load, the single-run comparison in the main evaluation reports 79.0% request-level SLO attainment for RAVEL and 65.6% for DualMap, a gain of 13.4 percentage points. The repeated-run study likewise uses DualMap as the fixed comparator for all three workloads and all five repetitions; it is a separate experimental batch from the single-run matrix.

The five-run DeepResearch study yields a mean paired gain of 17.40 percentage points over DualMap. This value is reported separately from the 13.4 percentage-point single-run

Table 6: Mean paired SLO-attainment gains at 16 $\times$ load across five repetitions. Intervals are two-sided 95% Student- t CIs over paired gains.

Workload	Mean (pp)	95% CI (pp)
LMSYS	4.08	[0.49, 7.67]
Burst	7.48	[3.78, 11.18]
DeepResearch	17.40	[10.56, 24.24]

result because the two statistics summarize different experimental repetitions.

C.2 Five-Run Experimental Protocol

For each workload $w \in \{\text{LMSYS}, \text{Burst}, \text{DeepResearch}\}$ at 16 $\times$ load, the robustness experiment consists of five complete paired replays. Within paired repetition j , RAVEL and the workload’s fixed comparator use the same request set and order, trace timing, random seed, serving endpoints, physical-WAN topology, model/engine configuration, cache reset procedure, and warm-up protocol.

For workload w and paired run j , define the SLO-attainment gain

$$\Delta_{w,j} = \text{SLOAttain}_{\text{RAVEL},w,j} - \text{SLOAttain}_{\text{DualMap},w,j}. \quad (41)$$

The five-run paired mean is

$$\bar{\Delta}_w = \frac{1}{5} \sum_{j=1}^5 \Delta_{w,j}. \quad (42)$$

Let

$$s_{\Delta,w} = \sqrt{\frac{1}{4} \sum_{j=1}^5 (\Delta_{w,j} - \bar{\Delta}_w)^2}. \quad (43)$$

We compute a two-sided 95% Student- t confidence interval over the five paired gains:

$$\bar{\Delta}_w \pm t_{0.975,4} \frac{s_{\Delta,w}}{\sqrt{5}}, \quad t_{0.975,4} = 2.776. \quad (44)$$

The pairing is performed at the replay level; we do not construct independent confidence intervals for RAVEL and the comparator and subtract them. The five paired gains (in percentage points) are LMSYS: {3.2, 2.4, 8.0, 0.8, 6.0}, Burst: {2.4, 7.2, 9.2, 9.4, 9.2}, and DeepResearch: {19.4, 18.6, 11.8, 25.0, 12.2}.

C.3 Repeated-Run Figure and Interpretation

The repeated-run figure plots one point per workload at $\bar{\Delta}_w$ with the 95% Student- t interval from Eq. 44 and a vertical reference line at zero. Each point summarizes five complete paired replays against DualMap.

A positive point estimate establishes that the mean paired difference is positive for those five repetitions. All three

95% intervals in Table 6 lie above zero: LMSYS improves by 4.08 pp on average (95% CI: 0.49–7.67 pp), Burst by 7.48 pp (3.78–11.18 pp), and DeepResearch by 17.40 pp (10.56–24.24 pp). These intervals quantify replay-level variation across the five paired repetitions; they are not formal per-request SLO guarantees.

C.4 Interpretation Boundaries

The result discussion observes the following boundaries:

- The 13.4 pp value is the single-run DeepResearch $16\times$ RAVEL–DualMap difference.
- The 17.40 pp value is the mean RAVEL–DualMap difference across five paired DeepResearch repetitions.
- The two statistics summarize different experimental repetitions and are therefore reported separately.
- Paired confidence intervals quantify replay-level uncertainty for the five-run experiment; they do not convert an empirical result into a formal SLO guarantee.